

6. 端到端示例

示例: 上传“智能投标产品手册.docx”并问“这份材料里有没有酒店行业案例？”

入库:

1. temp_upload -> temp_file_id
2. add_resource(wait=true, instruction=面向投标提取产品能力、案例、资质)
3. WordParser 转换内容，MarkdownParser 分片成章节树
4. TreeBuilder 生成 root_uri=viking://resources/投标资料/智能投标产品手册
5. SemanticDag 生成每个目录的 .abstract.md/.overview.md
6. VectorDB 写入 L0/L1/L2 向量记录

检索:

1. search(query, session_id) 读取会话摘要和最近消息
2. IntentAnalyzer 生成 resource 类型 TypedQuery
3. 全局召回命中“行业案例”相关 L0/L1
4. 递归进入该目录，召回酒店案例 L2 分片
5. rerank 提升与“酒店行业案例”最相关的分片
6. 返回 MatchedContext，供 RAG 生成答案并引用来源

跟踪对象	产生阶段	用途
root_uri	TreeBuilder / ResourceProcessor	正式知识树入口，检索时可作为 target_uri。
分片文件 URI	MarkdownParser/Parser	L2 内容节点，最终答案引用来源。
.abstract.md	SemanticDag	L0 快速召回文本。
.overview.md	SemanticDag	L1 目录导航和重排文本。
SemanticMsg.id	Summarizer/SemanticQueue	关联语义处理和 embedding 任务。
document_id	KnowledgeDocumentRegistry	查询 ready/failed 处理状态。
MatchedContext.score	HierarchicalRetriever	最终排序后的可解释检索分数。

7. 源码映射

主题	关键源码	说明
资源接入	openviking/server/routers/resources.py	temp_upload、add_resource 路由和请求参数。
入库编排	openviking/service/resource_service.py	URI 约束、文档登记、wait、watch task。
执行处理	openviking/utils/resource_processor.py	解析、TreeBuilder、正式落盘、Summarizer 投递。
来源路由	openviking/utils/media_processor.py	URL、目录、文件、ZIP、原始文本路由。
分片核心	openviking/parse/parsers/markdown.py	标题识别、小节合并、超大段落拆分、preview-order。
解析器选择	openviking/parse/registry.py	根据扩展名、URL 类型选择 parser。